LiveTix

Plataforma Global de Venta de Boletos

System Design Challenge

Autor: Bruno Moutinho

Rol: Cloud Engineer Senior

Fecha: 10 de October de 2026

Nube: AWS (3 regiones, 3 continentes)

Documento de Decisiones Arquitectónicas

Autor: Bruno Moutinho **Rol:** Cloud Engineer Senior **Nube elegida:** AWS **Regiones:** us-east-1 (Norteamérica), eu-west-1 (Europa), sa-east-1 (Sudamérica)

---

1. Resumen ejecutivo

LiveTix es una plataforma global de venta de boletos para eventos en vivo, operando en 4 países sobre 3 continentes. El reto central es soportar picos de 10-20x en segundos **sin vender el mismo asiento dos veces** y **sin cobrar sin entregar boleto**.

Decisiones clave:

- **AWS** como nube única (no multi-nube).
- **3 regiones activas** con datos regionales, catálogo global.
- **Serverless gestionado** (Fargate, Lambda, DynamoDB, Aurora) para operar con 5 ingenieros.
- **Patrón Saga + Outbox + Reconciliador** para garantizar consistencia pago↔boleto.
- **Sala de espera virtual + SQS FIFO** para absorber picos sin depender de autoescalado reactivo.

---

2. Manejo de picos (20x en un minuto)

Problema

El autoescalado reactivo (target tracking en ECS, por ejemplo) tarda 2-5 minutos en reaccionar. Un pico de 20x en 60 segundos satura el sistema antes de que escale.

Solución en capas

1. **Pre-escalado programado.** Sabemos cuándo salen los boletos (fecha/hora del "drop"). Con EventBridge + Lambda programamos capacidad 15 min antes subiendo el `desiredCount` de ECS.

2. **Sala de espera virtual.** Cuando el tráfico supera un umbral (ej. 5.000 req/s), el API Gateway devuelve un token de espera. El usuario ve "posición 4.231 en la fila". Convierte un pico de 20x en un flujo controlado de 1x. Implementado con contador en ElastiCache (Redis) por evento.

3. **SQS FIFO como buffer.** El checkout no procesa directo: encola. Los workers consumen a ritmo controlado. Si la cola crece, el usuario ve "procesando..." pero el sistema no se cae.

4. **Reserva con TTL de 10 min.** El asiento se reserva por 10 minutos. Si el usuario no paga, se libera. Evita bloqueos indefinidos.

5. **CloudFront + cache agresivo** para todo lo que no sea checkout (catálogo, imágenes, páginas estáticas). Reduce carga al origen ~80%.

Tradeoffs

- La sala de espera añade fricción al usuario. **Mitigación:** se activa solo cuando el tráfico supera el umbral.
- El pre-escalado consume capacidad extra que se paga aunque no se use. **Aceptable:** el costo de 30 min de capacidad extra es mínimo comparado con perder ventas por caída.

---

3. Concurrencia de inventario

Solución: reserva atómica con conditional write

En Aurora PostgreSQL:

```
UPDATE seats
SET status = 'RESERVED',
    order_id = $1,
    reserved_until = NOW() + INTERVAL '10 minutes'
WHERE seat_id = $2
  AND status = 'AVAILABLE';
-- Si affected_rows = 0 → alguien más lo tomó → devolver 409
```

En DynamoDB (alternativa):

```
UpdateItem
  Key: { eventId, seatId }
  ConditionExpression: attribute_not_exists(order_id)
  UpdateExpression: SET status = :reserved, order_id = :oid
```

Por qué funciona

La base de datos garantiza atomicidad en una sola fila. No hay race condition posible: dos transacciones concurrentes no pueden ambas ganar la misma fila.

Decisión: inventario por región

El inventario de un evento vive en **una sola región** (la "dueña", la más cercana al venue). Esto evita coordinación distribuida.

Tradeoffs

- Un usuario en Europa comprando un evento en Brasil tendrá ~150ms de latencia en el checkout. **Aceptable:** preferible a consistencia fuerte multi-región que cuesta 10x y es compleja de operar.
- La región dueña es un punto único de fallo para ese evento. **Mitigación:** si cae la región, el evento queda en modo "solo lectura" y se reactiva al volver. Los pedidos en vuelo se reconcilian.

---

4. Consistencia de pago (nunca cobrar sin boleto)

Patrón: Saga + Outbox + Idempotencia + Reconciliador

Idempotencia. Cada operación lleva un `idempotency_key` (UUID por orderId). Stripe lo soporta nativamente. Si reintentas, no cobra dos veces.

Outbox pattern. En la misma transacción de DB que actualiza `orders.status = PAID`, escribimos un registro en tabla `outbox`. Un worker lee el outbox y publica el evento. Garantiza que si la DB commitea, el evento se publica (at-least-once).

Saga. La compra es una secuencia de pasos con compensaciones:

- Si falla emisión de boleto → reembolso automático.
- Si falla pago → liberar reserva de asiento.

Reconciliador. Job cada minuto que busca inconsistencias:

- Orders en `PAID` sin `TICKET_ISSUED` > 2 min → reintentar emisión.
- Orders en `PAID` sin `TICKET_ISSUED` > 10 min → reembolso + alerta.
- Asientos `RESERVED` con `reserved_until` vencido → liberar.

Máquina de estados del pedido

```
PENDING → RESERVED → PAID → TICKET_ISSUED
                  ↘ EXPIRED
         ↘ FAILED
                  PAID → REFUND_PENDING → REFUNDED
```

Tradeoffs

- El outbox añade ~1-2 segundos de latencia entre pago y boleto. **Aceptable:** el usuario ya pagó, puede esperar unos segundos.
- El reconciliador puede tardar hasta 1 minuto en detectar una inconsistencia. **Aceptable:** es una red de seguridad, no el camino feliz.

---

5. Multi-región con control de costos

Qué se replica

- **Catálogo de eventos** (DynamoDB Global Tables) → lectura local.
- **Contenido estático** (S3 + CloudFront).
- **Configuración y feature flags**.

Qué NO se replica

- **Inventario activo** de un evento (vive en la región dueña).
- **Pedidos activos** (viven en la región donde se crearon).
- **Bases de datos de escritura** (cada región tiene la suya).

Enrutamiento

- Route 53 latency-based routing para lectura.
- Para checkout, el frontend sabe a qué región pertenece el evento y llama directo a esa región.

Tradeoffs

- Latencia cross-region en checkout (~150ms). **Aceptable.**
- Consistencia eventual del catálogo entre regiones (~1s). **Aceptable:** el catálogo no es crítico para consistencia.
- Costos: 3 regiones activas cuestan ~3x una sola. **Justificado:** el brief exige baja latencia global y tolerancia a caída regional.

---

6. Confiabilidad

Caída de zona (AZ)

Cada servicio corre en 3 AZs. Aurora Multi-AZ. Si cae una AZ, el balanceador redirige. **El cliente no lo nota.**

Caída de región completa

Route 53 health checks detectan la caída y redirigen a la región más cercana. Los eventos de esa región quedan en modo "solo lectura" hasta que vuelva. Los pedidos en vuelo se reconcilian al volver.

¿El cliente lo nota?

- Para lectura: no (CloudFront cachea).
- Para checkout de eventos en la región caída: sí, ve un mensaje "estamos teniendo problemas, intenta en unos minutos". Es preferible a cobrar sin poder entregar.

Tradeoffs

- No hay failover activo-activo de escritura. Cuesta mucho y para un equipo de 5 no es operable.
- RTO estimado: 5-10 min (detección + redirección Route 53).
- RPO: 0 para pedidos confirmados (Aurora Multi-AZ con sync replication).

---

7. Observabilidad

Tres pilares

Métricas (CloudWatch + Prometheus):

- `checkout_latency_p99`
- `orders_pending_count`
- `seats_reserved_expired`
- `payment_webhook_failures`
- `queue_depth_checkout`

Alarmas:

- `orders_pending > 1000` por 2 min → alerta.
- `payment_webhook_failures > 5` en 5 min → alerta.
- `queue_depth_checkout > 10000` por 5 min → alerta.

Trazas (X-Ray / OpenTelemetry):

- Cada compra tiene un `trace_id` que atraviesa API → checkout → inventario → pago → emisión.
- Permite ver exactamente dónde se atascó un pedido.
- Sampling: 10% normal, 100% checkout.

Logs estructurados (JSON):

- Cada log lleva `order_id`, `user_id`, `region`, `trace_id`.
- Búsqueda en CloudWatch Logs Insights.

Alertas proactivas

- Canary que compra un boleto de prueba cada 5 min en cada región.
- Si el canary falla → alerta antes que los usuarios.

Tradeoffs

- X-Ray tiene costo por traza muestreada. **Mitigación:** sampling inteligente.
- Las alarmas pueden generar fatiga si son muy sensibles. **Mitigación:** revisar thresholds mensualmente.

8. Lo que NO resolví (honestidad)

1. **Fraude y bots.** No cubrí detección de scalpers con miles de cuentas. Haría device fingerprinting + rate limiting por IP.
2. **Reembolsos y cancelaciones.** No diseñé el flujo completo de devolución. Haría una saga similar a la compra.
3. **Analítica en tiempo real.** Para el negocio sería valioso ver ventas por segundo. Fase 2 con Kinesis + Redshift.
4. **Cumplimiento fiscal por país.** Cada país tiene reglas de facturación distintas. Lo delegaría a Stripe Tax.
5. **Testing de caos.** No incluí chaos engineering. Con más tiempo, haría experimentos controlados de caída de región.
6. **Internacionalización.** Precios en monedas locales, idiomas, zonas horarias. Fase 2.

9. Conclusión

La arquitectura prioriza:

1. **Consistencia en el camino crítico** (pago↔boleto).
2. **Simplicidad operativa** (serverless gestionado, 5 ingenieros).
3. **Latencia global baja** (3 regiones, CDN, datos regionales).
4. **Resiliencia** (Multi-AZ, reconciliador, canary).

Cada decisión tiene un tradeoff explícito. No hay tecnología de moda: todo está justificado por costo, escala, consistencia o resiliencia.

Infraestructura como Código (Conceptual)

1. Estructura de carpetas

```
livetix-infra/
├── modules/                    # Módulos reutilizables
│   ├── network/                # VPC, subnets, security groups
│   ├── database/               # Aurora, DynamoDB
│   ├── compute/                # ECS service, Lambda
│   ├── queue/                  # SQS, SNS
│   └── observability/          # CloudWatch, alarms
├── environments/
│   ├── dev/
│   ├── staging/
│   └── prod/
│       ├── us-east-1/
│       ├── eu-west-1/
│       └── sa-east-1/
├── global/                     # Route 53, CloudFront, IAM
└── .github/
    └── workflows/
        ├── plan.yml
        └── apply.yml
```

2. Módulos: por qué y cómo

Cada módulo encapsula un recurso repetible. Ejemplo:

```
# modules/compute/main.tf
variable "service_name" { type = string }
variable "region"       { type = string }
variable "cpu"          { default = 512 }
variable "memory"       { default = 1024 }

resource "aws_ecs_service" "this" {
  name            = var.service_name
  cluster         = aws_ecs_cluster.this.id
  task_definition = aws_ecs_task_definition.this.arn
  desired_count   = 2
  launch_type     = "FARGATE"

  network_configuration {
    subnets         = var.private_subnets
    security_groups = [var.sg_id]
  }
}
```

Uso en `environments/prod/us-east-1/main.tf`:

```
module "checkout_service" {
  source       = "../../../modules/compute"
  service_name = "checkout"
  region       = "us-east-1"
  cpu          = 1024
  memory       = 2048
}
```

3. Ambientes y versionado

- **dev:** un desarrollador, datos sintéticos, se puede romper.
- **staging:** espejo de prod con 10% de capacidad.
- **prod:** multi-región, datos reales, cambios con aprobación.

Versionado:

- Código en Git (monorepo).
- Estado de Terraform en S3 + DynamoDB lock (uno por ambiente).
- Tags en Git: `v1.2.3` → despliegue a prod.
- Módulos versionados con tags.

4. Secretos y credenciales

- **AWS Secrets Manager** para API keys de Stripe, DB passwords.
- **IAM Roles** para que los servicios accedan sin credenciales estáticas.
- **Nunca** secretos en el repo. `terraform.tfvars` va en `.gitignore`.
- Rotación automática de secretos cada 90 días.
- En CI/CD, usar **OIDC** para que GitHub Actions asuma rol de AWS sin access keys.

  resource "aws_secretsmanager_secret" "stripe_key" { name = "livetix/prod/stripe/api_key" }

  resource "aws_secretsmanager_secret_version" "stripe_key" { secret_id = aws_secretsmanager_secret.stripe_key.id secret_string = var.stripe_api_key }

5. Pipeline CI/CD

```
# .github/workflows/deploy.yml
name: Deploy LiveTix

on:
  push:
    branches: [main]
    paths: ['environments/prod/**']
```

```yaml
jobs:
  plan:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/GitHubActionsDeploy
          aws-region: us-east-1
      - name: Terraform Init
        run: terraform -chdir=environments/prod/us-east-1 init
      - name: Terraform Plan
        run: terraform -chdir=environments/prod/us-east-1 plan -out=tfplan
      - name: Upload plan
        uses: actions/upload-artifact@v4
        with:
          name: tfplan
          path: environments/prod/us-east-1/tfplan

  apply:
    needs: plan
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/GitHubActionsDeploy
          aws-region: us-east-1
      - uses: actions/download-artifact@v4
        with:
          name: tfplan
      - name: Terraform Apply
        run: terraform -chdir=environments/prod/us-east-1 apply -auto-approve tfplan
```

Estrategia de despliegue:

- **Blue/Green** en ECS: nueva versión al 5% de tráfico (canary), si métricas OK se cambia al 100%.
- **Rollback automático** si `error_rate > 1%` en 5 min.

6. Tradeoffs

- **Monorepo vs multi-repo:** elegí monorepo por simplicidad para 5 ingenieros. Multi-repo sería mejor si creciera a 20+.

- **Terraform vs CDK:** elegí Terraform por ser agnóstico y tener mejor ecosistema. CDK es más expresivo pero acopla a AWS.
- **Estado remoto en S3:** obligatorio para trabajo en equipo. Tradeoff: requiere bootstrap inicial (S3 + DynamoDB lock).

Diagrama de Arquitectura

